

IT3051 Fundamentals of Data Mining

Mini Project 2026

Used Car Price Prediction System

for the Georgian Market

Technical Report

Group KND_11

Name	Registration number
N S Devapriya (Nirodha)	IT23557956
W G B S Wijerathna	IT23630734
R A N K Ranasinghe	IT23780088
T M A R Gunasekara	IT23614208

October 2026

Contents

1. Introduction and Problem Definition	4
1.1 Background	4
1.2 Problem Statement.....	4
1.3 Objectives	4
1.4 Scope.....	4
1.5 Structure of the Report.....	5
2. Assigned Scenario and User and Stakeholder Requirements	5
2.1 Scenario	5
2.2 Users and Decisions Supported.....	5
2.3 Data Mining Task.....	5
2.4 Requirements.....	5
2.5 Inputs and Outputs.....	6
3. Dataset Identification, Source, Citation and Validation	6
3.1 Source and Citation	6
3.2 Description and Real-World Context	7
3.3 Why the Dataset Suits the Scenario.....	7
3.4 Attributes.....	7
3.5 Data Quality Observations and Known Limitations	8
3.6 Ethical, Privacy and Licensing Considerations.....	8
4. Data Understanding and Exploratory Data Analysis	8
4.1 Structure and Variable Types.....	9
4.2 The Target Variable.....	9
4.3 Missing Values and Duplicates	9
4.4 Outliers and Unusual Observations	9
4.5 Relationships in the Data	10
4.6 Balance of the Categorical Variables	11
4.7 Data Leakage Risks	11
4.8 Observations That Shaped Later Decisions	12
5. Data Cleaning and Preprocessing	12
5.1 Cleaning Steps.....	12
5.2 Decisions Worth Justifying.....	13
6. Feature Engineering and Feature Selection	14
6.1 Engineered Features	14
6.2 Features That Were Not Used	14
6.3 Feature Selection	15
7. Algorithms Implemented and Their Rationale	15
8. Model Evaluation and Comparison.....	16
8.1 Validation Strategy and Metrics.....	16
8.2 Baseline Models	16
8.3 Interpretation	16
8.4 Where the Models Make Large Errors	17
9. Hyperparameter Tuning and Optimisation	17
9.1 Search Strategy.....	17

9.2 Selected Settings.....	18
9.3 Effect of Tuning	18
9.4 Feature and Target Choices.....	19
10. Final Model Selection and Justification	20
10.1 Selection Rule and Result	20
10.2 Justification	20
10.3 Behaviour of the Final Model	21
11. System Architecture and Implementation	21
11.1 Overview	21
11.2 Components.....	22
11.3 Technology.....	22
11.4 Data Flow of One Prediction.....	22
12. Backend and Frontend Development	23
12.1 Backend (Stage 9).....	23
12.2 Frontend (Stage 10)	24
12.3 Running the System.....	25
13. System Testing and Results	26
13.1 Automated Tests.....	26
13.2 Consistency Between Training and Serving	26
13.3 Manual Tests of the Page.....	26
13.4 Model Results in the System.....	27
14. Limitations and Possible Future Improvements	27
14.1 Limitations	27
14.2 Possible Improvements	28
15. Individual and Group Contributions	28
References	29

1. Introduction and Problem Definition

1.1 Background

Buying or selling a used car is hard to do well because the price depends on many things at the same time: how old the car is, how far it has been driven, the make, the engine, the fuel, the gear box and the equipment it carries. Online marketplaces in Georgia list thousands of used cars, and each seller chooses the asking price. A buyer who sees two similar cars at very different prices has no easy way to know which price is reasonable, and a seller has the opposite problem when deciding what to ask.

If we have a big collection of past listings, a model can learn how a car's details relate to its price and then guess the price of a car it hasn't seen before. In this project we build that kind of model and put it inside a simple web app, so anyone can use it without knowing anything about data mining.

1.2 Problem Statement

Given the specifications of a used car (production year, mileage, make, body type, engine, fuel, gear box, drive wheels, equipment and, optionally, the customs levy), estimate its market price in US dollars, and show the user how reliable that estimate is.

1.3 Objectives

1. Select a public dataset that fits the problem and have it validated by the instructor.
2. Understand, clean and prepare the data without leaking information from the test set into training.
3. Train and compare at least four regression algorithms using a sound validation strategy.
4. Tune the models with a proper search and choose the final model with a rule that does not use the test set.
5. Build a backend that validates input, applies exactly the same preprocessing as in training and returns a prediction, and a frontend that lets a user work with it.
6. Test the complete system and report its limitations honestly.

1.4 Scope

The system estimates a single price for one car at a time. It is trained on listings with production years from 1980 to 2020 and therefore only accepts cars in that range. It is an estimation aid and not a formal valuation. Value retention scoring, deal classification and price forecasting were considered during preprocessing but are outside the final system; the reason is explained in Section 2.5.

1.5 Structure of the Report

Sections 2 and 3 describe the scenario and the dataset. Sections 4 to 6 cover data understanding, cleaning and feature engineering. Sections 7 to 10 describe the algorithms, their evaluation, tuning and the final model choice. Sections 11 to 13 describe the system and how it was tested. Section 14 lists limitations and future work and Section 15 records the contribution of each member.

2. Assigned Scenario and User and Stakeholder Requirements

2.1 Scenario

The scenario chosen by the group is a web-based price estimation service for the Georgian used-car market. A user opens the page, enters the details of a car and receives an estimated price, a likely price range and, if they entered an asking price, a statement of how that asking price compares with the estimate.

2.2 Users and Decisions Supported

Table 1. Stakeholders and the decisions the system supports

Stakeholder	Need	How the system helps
Private buyer	Decide whether an advertised price is reasonable before contacting the seller.	Shows an estimate and range; the asking-price check says whether the advertised price is below, inside or above the range.
Private seller	Choose a sensible price to list a car at.	Gives an estimate and range from the car's specification.
Small dealer or trader	Appraise trade-in cars quickly and consistently.	Gives an instant estimate and shows what four different models predict, so a large disagreement is visible.

2.3 Data Mining Task

The task is **supervised regression**. The target variable is the listing price in US dollars (Price). Models are judged by R^2 , root mean squared error (RMSE) and mean absolute error (MAE) on cars that were not used for training.

2.4 Requirements

Table 2. Functional and non-functional requirements

ID	Requirement
FR1	The user can enter the specification of a car through a form with clear labels, units and allowed ranges.
FR2	The system checks every input and tells the user exactly which field is wrong and why.
FR3	The system applies the same preprocessing as in training before calling the model.
FR4	The system returns an estimated price in USD and a likely price range.

ID	Requirement
FR5	If the user gives an asking price, the system compares it with the estimate.
FR6	The user can see what each of the four trained models predicts and how accurate each model was on test data.
FR7	Missing optional input (the customs levy) is handled by a documented default and the user is told.
NFR1	Results appear in about a second on an ordinary computer.
NFR2	The page works on a normal desktop screen and on a phone-sized screen.
NFR3	The estimate is presented with its uncertainty and is not described as a guaranteed price.
NFR4	Training can be repeated from the saved scripts and gives the same results (fixed random seeds).

2.5 Inputs and Outputs

Table 3. Inputs and outputs of the system

Type	Items
Required inputs (15)	Manufacturer, production year, body type, colour, engine volume, turbocharged, cylinders, fuel type, gear box, drive wheels, steering wheel side, mileage, airbags, doors, leather interior.
Optional inputs (3)	Customs levy (USD), asking price (USD), prediction model (default is the final model).
Outputs	Estimated price (USD), likely price range, asking-price check, estimates and test accuracy of the four models, notices about defaults that were applied.

While preprocessing (Stage 4) we also created a value retention index and a three-way deal label (bargain, fair, overpriced) from the price. They are saved in the processed files, but we left them out of the final system. The reason is that our model's median error is about 17% of the price, which is bigger than the $\pm 15\%$ band we used to define the deal classes. A hard "bargain" or "overpriced" verdict would therefore be wrong quite often. Instead, the system just shows how the asking price compares with the estimated range and lets the user decide.

3. Dataset Identification, Source, Citation and Validation

3.1 Source and Citation

Table 4. Dataset summary

Item	Detail
Name	Car Price Prediction Challenge
Original source	Used-car listings from a Georgian online car marketplace (reported to be myauto.ge), published on Kaggle.
URL	https://www.kaggle.com/datasets/deepcontractor/car-price-prediction-challenge

Item	Detail
Citation	Contractor, D. (n.d.). Car Price Prediction Challenge [Data set]. Kaggle. https://www.kaggle.com/datasets/deepcontractor/car-price-prediction-challenge
File used	car_price_prediction.csv (kept as data/raw_car_price_prediction.csv)
Records and features	19,237 records, 18 columns
Target variable	Price (listing price in USD)
Task	Regression

3.2 Description and Real-World Context

Each row is one advertisement for a used car and describes the car as the seller entered it: make, model, production year, body category, engine volume, fuel, mileage, gear box, drive wheels, doors, steering wheel side, colour, airbags, leather interior and the customs levy. The price is what the seller asked, not necessarily what was finally paid. The data therefore describes the asking market of Georgia for the period in which the listings were collected; the newest production year is 2020.

3.3 Why the Dataset Suits the Scenario

- It contains the price of real listings, which is exactly the quantity the scenario needs to estimate.
- The features are the attributes that a buyer or seller knows about a car, so the same inputs can be requested from a user.
- It is large enough (19,237 records) for training, cross-validation and a separate test set, and it mixes numeric and categorical features.
- It needs real cleaning work (text-formatted numbers, corrupted values, missing values, extreme prices), which gives meaningful EDA and preprocessing.
- It is public and every group member can access it for academic use.

3.4 Attributes

Table 5. Raw attributes and the main quality issue found in each

Attribute	Type	Meaning	Quality observation
ID	Integer	Listing identifier	No predictive meaning; dropped.
Price	Integer	Asking price in USD (target)	Ranges from \$1 to \$26,307,500; many unrealistic values.
Levy	Text	Customs levy in USD	Contains "-" for 5,819 records (30.2%).
Manufacturer	Text	Make	65 distinct makes; a long tail of rare ones.
Model	Text	Model name	1,590 distinct values; too sparse to use directly.
Prod. year	Integer	Year of production	1939 to 2020.
Category	Text	Body type	11 categories.

Attribute	Type	Meaning	Quality observation
Leather interior	Text	Yes or No	Clean.
Fuel type	Text	Fuel	Seven values; Hydrogen occurs once.
Engine volume	Text	Volume in litres, sometimes with the word Turbo	Must be split into a number and a turbo flag.
Mileage	Text	Odometer, written like 12345 km	Must be converted to a number; extreme values.
Cylinders	Float	Number of cylinders	Strongly related to engine volume.
Gear box type	Text	Transmission	Four values.
Drive wheels	Text	Drive layout	Front, Rear, 4x4.
Doors	Text	Door count	Corrupted into dates: "04-May" (18,332 rows) and "02-Mar" (777) instead of 4-5 and 2-3.
Wheel	Text	Steering wheel side	Left wheel 17,753, right-hand drive 1,484.
Color	Text	Colour	16 values.
Airbags	Integer	Number of airbags	0 to 16.

3.5 Data Quality Observations and Known Limitations

- Extreme and unrealistic prices (from \$1 up to \$26,307,500) that distort any model trained on the raw data.
- 3,512 repeated listings once the ID column is ignored.
- Prices are asking prices set by sellers. Some very low prices on recent cars do not look like real sale prices, so the labels are noisy.
- Only 1.6% of the cleaned records are cars produced in 2019 or 2020, so the newest cars are poorly represented.
- The data comes from one country and one marketplace and reflects the market at the time of collection.

3.6 Ethical, Privacy and Licensing Considerations

The records describe cars only. They contain no names, telephone numbers, addresses or registration plates, so there is no personal data to protect. The data was collected from a public website and shared on Kaggle; the licence terms shown on the dataset page apply and the data is used here for academic purposes only. The model reflects one market and one period, so its estimates should not be treated as a formal appraisal or used for lending decisions.

4. Data Understanding and Exploratory Data Analysis

The exploration below was done on the raw data and then on the cleaned data. The code is in the Stage 3 part of the project notebook

(`source_stages_1_to_8/stages_1_to_5_eda_and_preprocessing.ipynb`) and the figures in this report were redrawn by `report/make_figures.py`.

4.1 Structure and Variable Types

The raw file has 19,237 rows and 18 columns. Seven columns are numeric after conversion (price, levy, production year, engine volume, mileage, cylinders, airbags) and the rest are categorical. Three numeric fields are stored as text and need conversion: levy (because of the "-" entries), mileage (the unit "km" is part of the text) and engine volume (the word "Turbo" is appended for turbocharged engines).

4.2 The Target Variable

The raw price is extremely skewed (skewness 136.5). The median is \$13,172, but the mean is \$18,556 and the maximum is \$26,307,500. The left histogram in Figure 1 shows a large block of listings priced at almost nothing, which are not real prices. After restricting prices to the range \$500 to \$150,000 (Section 5) the median is \$15,681, the middle half of the prices lies between \$9,095 and \$24,462, and skewness falls to 2.32. Taking the logarithm of the price removes the right tail but leaves a small group of very cheap cars on the left (right histogram), which is a first sign that some low prices are unreliable.

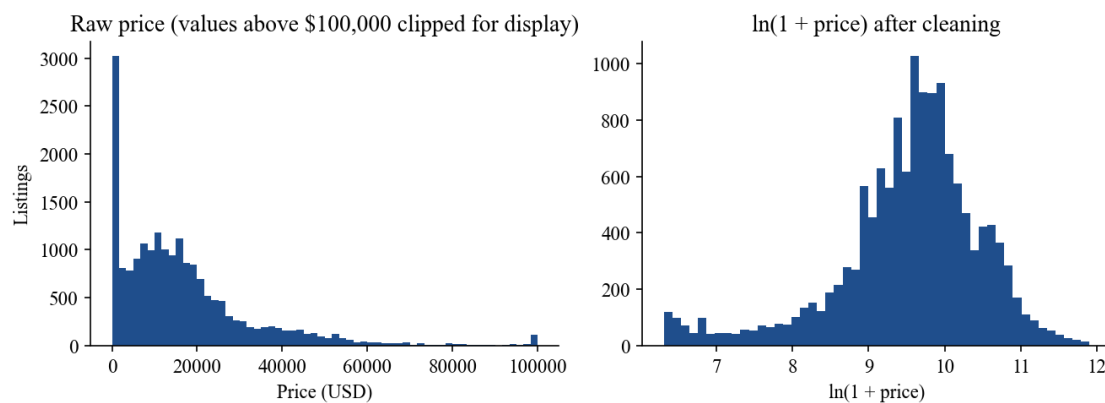


Figure 1. Distribution of the price before and after cleaning.

4.3 Missing Values and Duplicates

Only the levy column has missing values, coded as "-": 5,819 of 19,237 records (30.2%). After cleaning, 4,900 records (35%) are still missing a levy, so it cannot simply be dropped. The levy is a customs charge that depends on the make, engine and age of the car, so it is imputed from similar cars (Section 5).

Duplicates need care. Only 313 rows are exact duplicates including the ID, but 3,512 rows are identical once the ID is ignored, meaning the same car was advertised more than once. Keeping them would make the model see the same car several times and would let identical cars appear in both training and test data, so repeated rows were removed.

4.4 Outliers and Unusual Observations

Four kinds of unrealistic value were found: prices below \$500 or above \$150,000 (951 records after removing duplicates), implausible odometer readings below 10 km or above 500,000 km (824), engine volumes outside 0.5 to 7.0 litres (33) and production years outside 1980 to 2026 (22). A record can break more than one rule. The limits were chosen from what is realistic for

passenger cars on this market, not from the statistics of the sample, so that genuinely expensive cars are not removed just because they are rare.

4.5 Relationships in the Data

Age and price. Correlation with price is strongest for age (-0.40), followed by mileage (-0.27) and engine volume (0.21). Levy (0.08) and airbags (0.03) have almost no linear relationship with price. These values are small, which already suggests that price depends on combinations of features and not on any one of them.

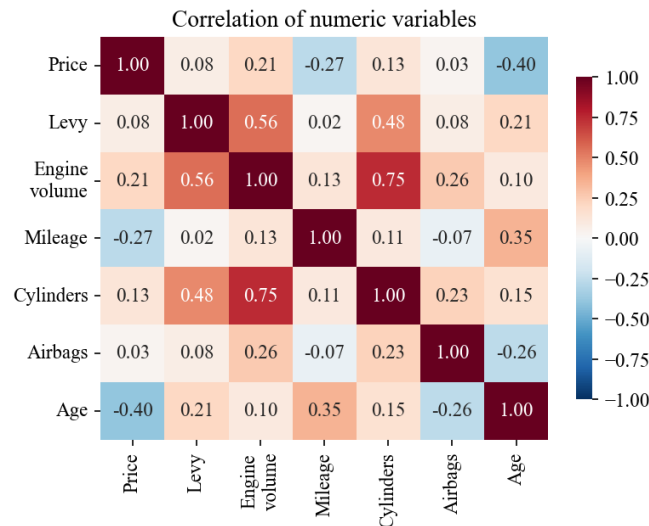


Figure 2. Correlation between the numeric variables.

Collinearity. Engine volume and the number of cylinders are strongly correlated (0.75), and age and mileage are moderately correlated (0.35). This can cause problems for plain linear regression, which is one reason we chose a regularised linear model (Ridge) as our linear baseline.

Steering wheel side. Right-hand-drive cars are a small group (1,237 of 14,011 cleaned records) but sell for much less: the mean price is \$9,887 against \$20,754 for left-hand drive (52% lower), and the medians are \$9,095 and \$16,741. Steering wheel side is therefore kept as a feature.

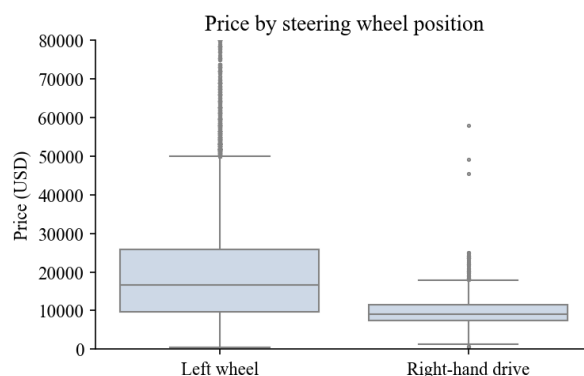


Figure 3. Price of left-hand-drive and right-hand-drive cars.

Age and fuel type. Prices fall as cars get older for all fuel types (Figure 4). For the youngest cars (age 6 to 8 years) the average price varies a lot between fuel types, because there are few such cars and a large share of them are expensive makes. The diesel line starts far above the others for this reason and not because diesel engines are worth more in general. Luxury makes have a higher average price (\$24,472) than the other makes (\$18,667).

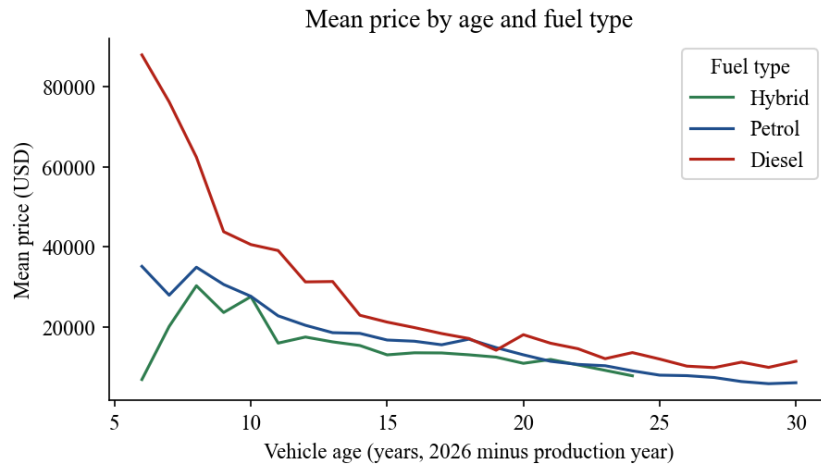


Figure 4. Mean price by age and fuel type.

4.6 Balance of the Categorical Variables

Class imbalance in the classification sense does not apply because the target is numeric. The categorical features are unevenly spread, however. Petrol (7,859), diesel (3,033) and hybrid (2,065) cars make up almost all records while LPG, CNG, plug-in hybrid and hydrogen are rare (the hydrogen category has a single car). Makes and colours also have long tails. This is handled by grouping rare makes and colours (Section 6) and by allowing the encoder to ignore categories it did not see in training.

4.7 Data Leakage Risks

Table 6. Leakage risks identified and how each is prevented

Risk	Prevention
The ID column identifies a listing and could let a model memorise cars.	Dropped before any modelling.
Test cars influencing the scaler, the encoder or the imputation values.	The data is split first. Scaling, encoding and levy medians are fitted on the training split only and then applied to the test split.
The same car appearing in training and test through repeated listings.	Repeated rows removed before the split. A check afterwards still found 73 test rows with the same features as a training row (different listings of the same specification); this is about 2.6% of the test set and removing them changes test R^2 from 0.7628 to 0.7667, so the effect is small.
Targets derived from the price (value retention index, tax overhead ratio, deal class) being used as inputs.	They are stored with the targets and are excluded from the feature matrix.

Risk	Prevention
The levy is a customs charge that may carry some information about the car's value.	Its correlation with price is only 0.08. It is optional in the application, and the ablation test in Section 9.4 shows what the model loses without it.
Hyperparameters and the final model chosen by looking at the test set.	All tuning uses cross-validation inside the training split; the test split is used once for reporting and is not used to choose the model.

4.8 Observations That Shaped Later Decisions

Table 7. From observation to decision

Observation	Decision
Unrealistic prices, mileage, engine volume and years.	Domain-range filtering (Section 5).
Numbers stored as text; Doors corrupted into dates.	Parsing and a mapping back to 4-5 and 2-3.
35% of levies missing after cleaning.	Imputation by the training median of the make.
Repeated listings.	Duplicates removed before the split.
Strong relationship between price and age, mileage, make and wheel side; weak linear correlations.	Engineer age, mileage per year and flags; use non-linear models.
Long tails in make and colour.	Group the rare values as Other.
Collinear numeric variables.	Include a regularised linear model; rely on trees for the rest.
Few cars produced in 2019 and 2020.	Accept only years 1980 to 2020 in the application and report the weakness (Section 14).

5. Data Cleaning and Preprocessing

5.1 Cleaning Steps

Table 8. Cleaning and preprocessing steps in the order they were applied

Step	Action	Reason
1	Drop ID	No predictive meaning; avoids memorising listings.
2	Remove repeated listings (3,512 rows; 19,237 to 15,725)	Same car counted more than once.
3	Convert Mileage to a number by removing " km"	Needed for modelling.
4	Split Engine volume into a number (Engine_displacement) and a flag (Is_Turbo)	Two separate pieces of information in one text field.
5	Map "04-May" to "4-5" and "02-Mar" to "2-3" in Doors	Spreadsheet software had turned the ranges into dates.
6	Treat "-" in Levy as missing	It means the value is unknown.
7	Keep records with price \$500 to \$150,000, mileage 10 to 500,000 km, engine 0.5 to 7.0 L, year 1980 to 2026 (15,725 to 14,011 rows)	Remove unrealistic values (Section 4.4).
8	Split into 11,208 training (80%) and 2,803 test (20%) records, seed 42	Hold-out set for the final evaluation.

Step	Action	Reason
9	Impute the missing levy with the training median of the make, or the overall training median if the make has none	Uses information from training data only.
10	Scale 7 numeric features (standardisation), one-hot encode 9 categorical features, pass 2 binary flags through; fitted on the training split only	Required by the linear model and keeps the test set unseen.

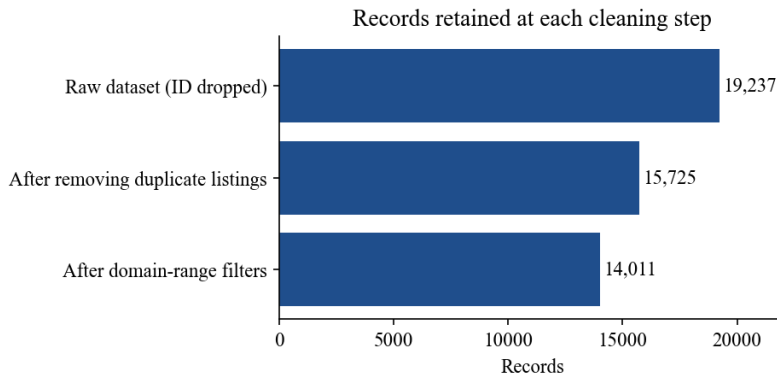


Figure 5. Number of records after each cleaning step.

In total 14,011 of the 19,237 raw records (72.8%) were kept. Most of the loss comes from the repeated listings and the unrealistic prices. The levy imputation uses the training median, which for the whole training set is \$779.

5.2 Decisions Worth Justifying

- We removed outliers using fixed limits instead of an automatic rule like the IQR. Cars above \$100,000 are rare, but they are real, and an IQR rule would have deleted valid luxury cars just because there are few of them.
- We filled missing levy values with the median levy of the same make. The levy depends on the kind of car, so using the make's median is a better guess than one single value for every car.
- We split the data into training and test sets first, and only then calculated the medians and fitted the scaler and encoder. This way nothing from the test set influences the training.
- The fitted transformer is saved as `data/preprocessing_pipeline.joblib`, and the backend loads the same file. So the app prepares a new car exactly the way the training data was prepared.
- After preprocessing, the training matrix has 11,208 rows and 57 feature columns, and the test matrix has 2,803 rows. Neither has any missing values.

6. Feature Engineering and Feature Selection

6.1 Engineered Features

Table 9. Features used by the models

Feature	Kind	How it is built	Why
Age	Numeric	2026 minus production year	Price falls with age; easier to read than the year.
Mileage	Numeric	Odometer in km	Wear on the car.
Mileage_per_year	Numeric	Mileage divided by (Age + 1)	Separates a car that is driven hard from one that is old but little used.
Engine_displacement	Numeric	Number part of Engine volume	Engine size.
Cylinders, Airbags, Levy	Numeric	As recorded (levy imputed)	Power, equipment and import cost.
Is_Turbo	Binary	1 if the engine text contains Turbo	Turbocharged engines tend to cost more.
Is_Luxury_Make	Binary	1 for 12 premium makes (Mercedes-Benz, BMW, Lexus, Porsche, Audi, Land Rover, Jaguar, Maserati, Bentley, Ferrari, Aston Martin, Cadillac)	Gives rare premium makes a shared signal even when the make itself is grouped as Other.
Manufacturer_grouped	Categorical	15 most frequent makes in the training split, the rest as Other	Avoids dozens of almost empty columns.
Color_grouped	Categorical	8 most frequent colours, the rest as Other	Same reason.
Category, Leather interior, Fuel type, Gear box type, Drive wheels, Doors, Wheel	Categorical	One-hot encoded, first level dropped	Models need numbers.

Together these give **57 input columns** after encoding. The age is measured from 2026, the year the project was done. Because every car is shifted by the same amount this does not change the ordering of the cars, and the same reference year is used by the application.

6.2 Features That Were Not Used

- ID: no meaning for prediction.
- Model: 1,590 distinct values, most with very few cars. Using it would cause overfitting, so it is not an input of the system. The model name is also not asked from the user.
- Prod. year: replaced by Age to avoid two copies of the same information.
- Raw Engine volume text: replaced by the numeric part and the turbo flag.
- Value retention index, tax overhead ratio and deal class: derived from the price, so they would leak the target.

6.3 Feature Selection

Feature selection was done in three ways. First, features were removed on principle, as listed above. Second, groups of features were removed one at a time from the final model and the cross-validated score was compared (Section 9.4). Third, the importance values of the final model were inspected.

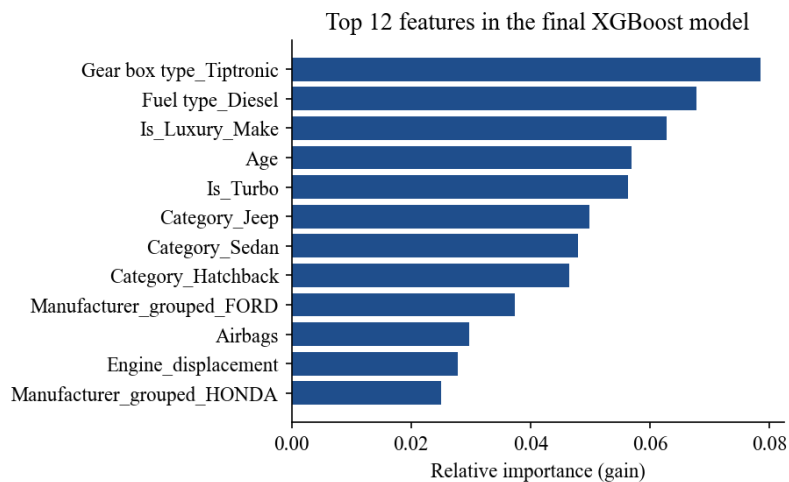


Figure 6. The twelve most important features of the final XGBoost model.

The importance chart shows that the information is spread across many features. Gear box type, fuel type, luxury make, age, turbo, body type and make all contribute, and no single feature dominates. None of the features we engineered turned out to be useless either. In the ablation tests, removing any group of features never clearly improved the model, so we kept all 57. Removing the colour columns or the mileage-per-year feature changed the scores by less than the normal fold-to-fold variation, so keeping them costs nothing in accuracy.

7. Algorithms Implemented and Their Rationale

We implemented four algorithms from different families so we could see what kind of model suits this problem. All of them predict the price directly. Tree models don't need scaled inputs, but we gave all four the same processed data so the comparison is fair.

Table 10. The four algorithms and why they were chosen

Algorithm	Family	Idea	Why it is included
Ridge Regression	Regularised linear	Fits a straight-line relationship and shrinks the coefficients with an L2 penalty so correlated inputs do not make them unstable.	A simple, fast baseline. If it did well, a complex model would not be needed; it also handles the correlated numeric features better than ordinary least squares.
Random Forest	Bagged trees	Averages many decision trees, each built on a random sample of cars and a random subset of features.	Captures non-linear relationships and interactions, resists overfitting and needs little tuning.

Algorithm	Family	Idea	Why it is included
LightGBM	Gradient boosting (histogram)	Builds trees one after another, each correcting the errors of the previous ones, using binned feature values and leaf-wise growth.	Boosting usually beats bagging on tabular data and LightGBM is fast.
XGBoost	Gradient boosting (regularised)	Boosted trees with explicit penalties on tree size and leaf values and with row and column subsampling.	Strong tabular-data method with fine control over overfitting.

Before running the experiments, we expected the three tree methods to give similar results and all be clearly better than Ridge. Used-car prices depend on combinations of features. For example, a luxury car loses value with age in a different way than an economy car does. A straight-line model can't capture that.

8. Model Evaluation and Comparison

8.1 Validation Strategy and Metrics

The 11,208 training records were used with **5-fold cross-validation** (shuffled, seed 42). Each model is trained five times on four fifths of the training data and scored on the remaining fifth; the mean and standard deviation of the five scores show how stable the model is. The 2,803 test records were not touched until the end and are used only to report the final performance. Three metrics are reported:

- **R²** is the share of the variation in price that the model explains (1 is perfect).
- **RMSE** is the typical error in dollars, giving extra weight to large mistakes.
- **MAE** is the average absolute error in dollars and is easy to explain to a non-technical user.

8.2 Baseline Models

The baselines use reasonable default settings, not tuned values: Ridge with alpha 10; Random Forest with 100 trees, depth 15 and a minimum leaf size of 2; LightGBM with 150 trees, learning rate 0.08 and 31 leaves; XGBoost with 150 trees, depth 6 and learning rate 0.08.

Table 11. Baseline results (CV on the training split, metrics on the test split)

Model	CV R ² (mean ± sd)	Test R ²	Test RMSE	Test MAE
Ridge Regression	0.4303 (± 0.0073)	0.4312	\$12,621	\$8,321
Random Forest	0.7331 (± 0.0201)	0.7437	\$8,472	\$4,863
LightGBM	0.7457 (± 0.0134)	0.7553	\$8,279	\$4,850
XGBoost	0.7394 (± 0.0166)	0.7503	\$8,363	\$4,890

8.3 Interpretation

The three tree-based models reach a cross-validated R² of about 0.73 to 0.75, while Ridge only reaches 0.43, so the straight-line model explains less than half of the variation in price. We

think this is because the effect of age, mileage or engine size isn't a fixed number of dollars. It changes with the make, body type and fuel, and trees can handle these interactions while a linear model can't. Between the tree models, the two boosting methods are slightly ahead of Random Forest, which matches the usual finding that boosting reduces bias that a forest of fully grown trees keeps.

The standard deviation over the folds (about 0.01 to 0.02) is larger than most differences between the three tree models. The comparison between them therefore has to be read with care; Section 10 returns to this point.

8.4 Where the Models Make Large Errors

The overall accuracy hides big differences between cheap and expensive cars. Table 12 shows the final model's errors by price band on the test set. In dollars, the error is smallest for mid-priced cars and largest for cars above \$30,000. In percentage terms, the error is largest for the cheapest cars: for cars under \$3,000 the median error is more than three times the price. Most of these are very cheap listings whose price doesn't follow the specification (Section 4.2), so the model can't learn them.

Table 12. Error of the final model by actual price band (test set)

Actual price	Cars	MAE	Median absolute % error
under 3k	190	\$6,020	355%
3k-8k	384	\$2,807	26%
8k-15k	737	\$2,768	17%
15k-30k	990	\$3,860	13%
over 30k	502	\$10,076	15%

Across all test cars the median absolute error is 17% of the price; 45% of the estimates are within 15% of the real price and 64% within 25%.

9. Hyperparameter Tuning and Optimisation

9.1 Search Strategy

Ridge has only one setting, the penalty alpha, so we tuned it with RidgeCV over 30 values between 0.01 and 10,000 on a logarithmic scale. The three tree models have many settings, so a full grid search would have been very large. Instead we used randomised search, which tries a fixed number of random combinations from the search space. It usually finds good settings with far fewer trials than a grid when only a few of the settings really matter. We scored every candidate with the same 5-fold cross-validation on the training split, then refitted the best one on the whole training split. The test split was not used in the search.

Table 13. Search spaces

Model	Candidates x folds	Search space
Random Forest	10 x 5	n_estimators {150, 250}; max_depth {12, 16, 20}; min_samples_leaf {2, 3, 5}; min_samples_split {4, 8}; max_features {0.5, 0.7, 0.9}
LightGBM	30 x 5	n_estimators {200, 400, 600}; learning_rate {0.02, 0.04, 0.08}; num_leaves {15, 31, 63}; min_child_samples {10, 20, 40}; subsample {0.7, 0.85, 1.0} with bagging every iteration; colsample_bytree {0.6, 0.8, 1.0}; reg_lambda {0, 1, 5}
XGBoost	25 x 5	n_estimators {300, 500, 800}; max_depth {4, 6, 8, 10}; learning_rate {0.02, 0.04, 0.08}; subsample {0.7, 0.85, 1.0}; colsample_bytree {0.6, 0.8, 1.0}; min_child_weight {1, 3, 5}; reg_lambda {1, 3, 10}
Ridge	30 alphas, 5 folds	alpha from 0.01 to 10,000 (logarithmic)

9.2 Selected Settings

Table 14. Best settings found by the search

Model	Best settings
Ridge Regression	alpha = 1.89
Random Forest	n_estimators = 150, min_samples_split = 8, min_samples_leaf = 2, max_features = 0.9, max_depth = 16
LightGBM	subsample_freq = 1, subsample = 1, reg_lambda = 0, num_leaves = 31, n_estimators = 200, min_child_samples = 10, learning_rate = 0.08, colsample_bytree = 0.8
XGBoost	subsample = 0.7, reg_lambda = 1, n_estimators = 500, min_child_weight = 5, max_depth = 8, learning_rate = 0.02, colsample_bytree = 1

9.3 Effect of Tuning

Table 15. Baseline compared with tuned models

Model	CV R ² baseline	CV R ² tuned (± sd)	Test R ² baseline	Test R ² tuned	Test MAE tuned
Ridge Regression	0.4303	0.4304 (± 0.0071)	0.4312	0.4318	\$8,330
Random Forest	0.7331	0.7334 (± 0.0194)	0.7437	0.7477	\$4,803
LightGBM	0.7457	0.7512 (± 0.0195)	0.7553	0.7597	\$4,819
XGBoost	0.7394	0.7525 (± 0.0192)	0.7503	0.7628	\$4,688

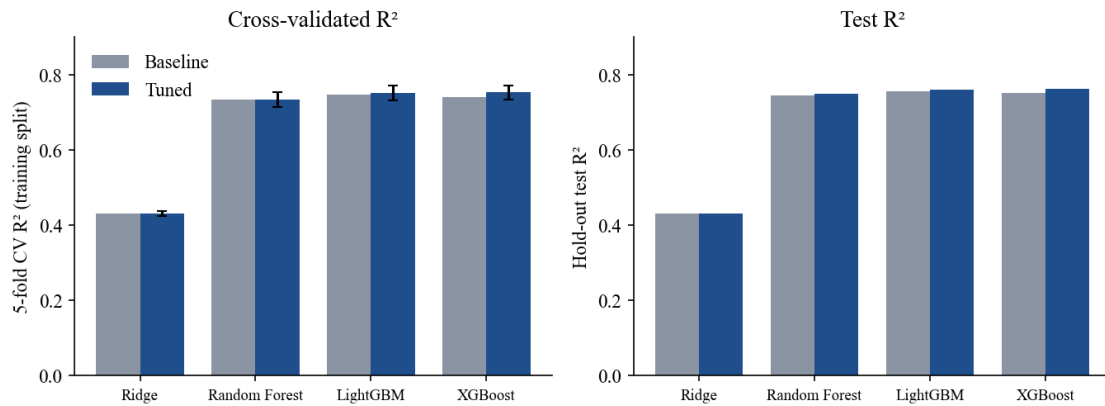


Figure 7. Baseline and tuned models: cross-validated R^2 (left) and test R^2 (right).

Tuning helped the boosted models a little. The cross-validated R^2 of XGBoost went from 0.7394 to 0.7525, and LightGBM went from 0.7457 to 0.7512. Random Forest and Ridge hardly changed. The gain is small because the default settings were already reasonable. Most of the improvement over the baseline comes from choosing the right type of model, not from the settings.

9.4 Feature and Target Choices

To check whether feature selection, feature engineering or a different target would improve the final model, the tuned XGBoost settings were re-run with parts of the feature set removed and with the log of the price as the target (`scripts/ablation_experiments.py`).

Table 16. Ablation results for XGBoost (CV on the training split, test split for reference)

Configuration	Features	CV R^2	Test R^2	Test MAE
All 57 features (final configuration)	57	0.7525	0.7628	\$4,688
Without Levy	56	0.7463	0.7539	\$4,800
Without Mileage_per_year	56	0.7548	0.7609	\$4,694
Without Is_Turbo and Is_Luxury_Make	55	0.7351	0.7228	\$4,926
Without colour dummies	49	0.7530	0.7598	\$4,698
Without Age and Mileage_per_year	55	0.6500	0.6453	\$5,859
Log-price target (all features)	57	0.7096	0.7088	\$4,860

- **Age and mileage matter most.** When we removed Age and Mileage_per_year, the CV R^2 dropped from 0.7525 to 0.6500.
- **The luxury and turbo flags help.** Without them, the CV R^2 falls to 0.7351 and the test R^2 to 0.7228, so these flags are worth keeping.
- **Levy helps a little.** Without it CV R^2 is 0.7463. The levy is optional in the application, so a user who does not know it still gets an estimate, with slightly lower expected accuracy.
- **Mileage per year and colour make no real difference** (changes within the fold-to-fold variation) and are kept.

- **Predicting the logarithm of the price was worse** (CV R^2 0.7096 against 0.7525), so the model predicts the price directly.

10. Final Model Selection and Justification

10.1 Selection Rule and Result

The final model was chosen by a rule fixed before looking at the test results: **the tuned model with the highest cross-validated R^2 on the training split**. Using the test set to choose would make the reported test score optimistic. The rule selects **XGBoost**.

Table 17. Tuned models ranked by the selection rule

Rank	Model	CV R^2 ($\pm$ sd)	Test R^2	Test RMSE	Test MAE
1	XGBoost	0.7525 ($\pm$ 0.0192)	0.7628	\$8,150	\$4,688
2	LightGBM	0.7512 ($\pm$ 0.0195)	0.7597	\$8,204	\$4,819
3	Random Forest	0.7334 ($\pm$ 0.0194)	0.7477	\$8,406	\$4,803
4	Ridge Regression	0.4304 ($\pm$ 0.0071)	0.4431	\$12,489	\$8,167

10.2 Justification

- **Best on the agreed criterion.** XGBoost has the highest cross-validated R^2 (0.7525). It also has the highest test R^2 (0.7628), the lowest RMSE (\$8,150) and the lowest MAE (\$4,688), so both ways of measuring pick the same model.
- **The tree models are close.** The gap between XGBoost and LightGBM in CV R^2 is only 0.0013, which is much smaller than the fold-to-fold standard deviation of about 0.02. So we can't say XGBoost is clearly better than LightGBM. It is the best by our selection rule and no worse on any metric. What is clear is that all the tree models are far better than Ridge (R^2 0.44).
- **Practical reasons.** The saved XGBoost model is only about 1.3 MB (the Random Forest is about 8.4 MB). It predicts one car in milliseconds and handles our mix of numeric and one-hot features without extra work.
- **Regularisation.** Row and column subsampling and the L2 penalty on the leaf values help reduce overfitting on rare makes and trims.

10.3 Behaviour of the Final Model

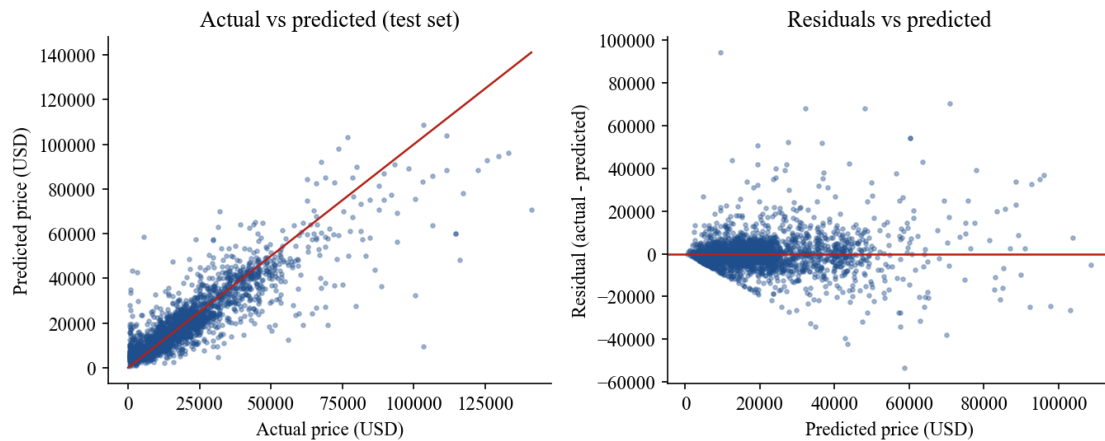


Figure 8. Final model on the test set: actual against predicted price (left) and residuals (right).

The points in the left chart follow the diagonal, which is the line of perfect prediction, and scatter more widely for expensive cars. The right chart shows that the size of the error grows with the price, so the model is more uncertain for expensive cars in dollar terms. The model also tends to predict too low for the most expensive cars. To describe this uncertainty in the application, the estimate is shown with a **likely range**. The range was built from the test set: for 80% of the test cars the real price was between 0.565 and 1.382 times the model's estimate, so the application shows the estimate multiplied by these two factors. The range is wide, and it is shown to the user as it is.

11. System Architecture and Implementation

11.1 Overview

The system has two parts that talk through a small JSON interface. The browser shows the form and the result. A Flask server receives the request, validates the input, rebuilds the training features, asks the models for a price and returns the answer. No database is needed because nothing is stored.

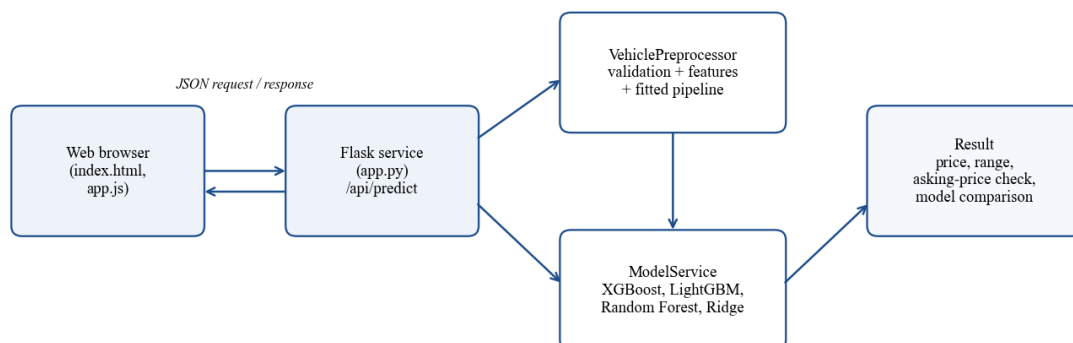


Figure 9. Architecture of the prediction system.

11.2 Components

Table 18. Main components and files

Component	File	Responsibility
Web page	templates/index.html, static/css/style.css	The input form and the result panel.
Front-end logic	static/js/app.js	Loads dropdown values, checks input before sending, calls the API, shows errors and results.
Web service	app.py	Routes and JSON responses; error handling.
Preprocessing	backend/preprocessor.py	Validation, feature building and the fitted transformer.
Model service	backend/model_service.py	Loads the four models and evaluation numbers, predicts, builds the price range.
Artifacts	models/*.joblib, data/*.joblib, data/*.json	Trained models, fitted pipeline, levy medians, dropdown values and metrics.
Tests	tests/test_api.py	Automated tests of the service.

11.3 Technology

Python 3 with pandas and NumPy for data handling, scikit-learn for the preprocessing pipeline, Ridge and Random Forest, XGBoost and LightGBM for the boosted models, joblib for saving models, Flask for the web service, and plain HTML, CSS and JavaScript for the page. The package versions are pinned in `requirements.txt` because saved models can only be loaded safely with the library versions that created them.

11.4 Data Flow of One Prediction

1. The user fills in the form. The browser checks required fields, whole numbers and ranges and shows messages next to the fields that are wrong.
2. The browser sends the values as JSON to `POST /api/predict`.
3. The server checks every field again (the browser cannot be trusted) and returns the error messages if anything is wrong.
4. The server builds the same features as in training: age, mileage per year, grouped make and colour, luxury and turbo flags; it fills a missing levy with the training median of the make.
5. The saved transformer scales and encodes the features into the 57 columns the models expect.
6. The selected model predicts the price; the other three models are also run for the comparison table.
7. The server returns the estimate, the range, the asking-price check, the model comparison and any notices, and the browser displays them.

12. Backend and Frontend Development

12.1 Backend (Stage 9)

Interface

Table 19. Service endpoints

Endpoint	Method	Purpose
/	GET	The web page.
/api/options	GET	Values for the dropdowns and the allowed production years.
/api/models	GET	The four models with cross-validated and test scores, the final model and the range factors.
/api/health	GET	Simple check that the service is running.
/api/predict	POST	Validates the car, predicts the price and returns the result. Returns status 400 with a message per field if the input is invalid, and 500 with a general message if the prediction itself fails.

Input validation

All validation is in `VehiclePreprocessor.validate`. Invalid input is rejected with a message for each wrong field; the server never replaces a wrong value with a default without telling the user. The only automatic default is the optional levy.

Table 20. Validation rules

Field	Rule
Production year	Required; whole number from 1980 to 2020 (the range of the training data).
Mileage	Required; number from 0 to 1,000,000 km. A trailing km and thousands separators are accepted.
Engine volume	Required; number from 0.5 to 7.0 litres.
Cylinders, airbags	Required; whole numbers from 1 to 16 and from 0 to 16.
Customs levy	Optional; 0 to 20,000. If empty, the training median of the make is used and a notice is shown.
Asking price	Optional; 1 to 1,000,000.
Manufacturer, colour, body type, fuel, gear box, drive wheels, doors, steering wheel, leather	Required; must be one of the listed options (case does not matter).
Turbocharged	True or false.
Prediction model	Optional; one of the four models, default is the final model.
Any number field	Text, empty values, nan and infinite values are rejected.

Preprocessing and prediction

The preprocessor rebuilds the 18 input columns used in training and applies the saved `ColumnTransformer`, so scaling and one-hot encoding use exactly the values learned from the training split. Makes and colours not among the frequent ones in training are mapped to `Other`,

as in training. The predicted price is limited to \$500 to \$150,000, which is the range of prices the models were trained on.

The estimate range is the estimate multiplied by 0.565 and 1.382. If an asking price is given, the response says whether it is below, inside or above this range and by how many per cent it differs from the estimate.

12.2 Frontend (Stage 10)

The page is a single screen with the form on the left and the result on the right. On a phone the two stack on top of each other. We kept the design plain on purpose: dark text on a light background, one accent colour, and no icons or animations, so the user can focus on the information.

- **Clear inputs.** The fields are grouped under four headings (vehicle, engine and drivetrain, condition and equipment, optional). Every label shows its unit, required fields have an asterisk, and a short hint under each field gives the allowed range.
- **Dropdowns instead of free text** for all categorical fields, so users can't make spelling mistakes. The options come from the server.
- **Validation.** The browser checks required fields, whole numbers and ranges before sending, and the server repeats the checks. Messages are shown beside the field and the first wrong field gets the focus. Server-side messages are shown in the same way.
- **Clear result.** The estimated price is the biggest item on the page. Below it come the range, a plain sentence about the asking price, any notices (for example, that a typical levy was used) and a table comparing the four models.
- **Honesty about uncertainty.** Under the price we explain that the range is the interval that held the real price for 80% of the test cars.
- **Connection errors.** If the server cannot be reached, a message tells the user to check that the application is running.

Used Car Price Prediction

Enter the details of a used car to get an estimated market price in US dollars. Estimates are based on car listings from Georgia (the country) with production years 1980–2020.

Vehicle details

Fields marked * are required.

Vehicle

Manufacturer *
HYUNDAI

Production year *
2012
Between 1980 and 2020

Body type *
Sedan

Colour *
Silver

Engine and drivetrain

Engine volume (litres) *
2
0.5 to 7.0, for example 2.0

Cylinders *
4

☐ Turbocharged

Fuel type *
Petrol

Gear box *
Automatic

Drive wheels *
Front

Steering wheel *
Left wheel

Condition and equipment

Estimate

2012 HYUNDAI

\$16,565

Likely range: \$9,360 to \$22,894
Estimated with XGBoost. The range is where the real price fell for 80% of the test cars.

Customs levy was not provided; the training-set median for this make (\$770) was used.

Asking price check

The asking price of \$7,500 is 54.7% below the estimate. This is lower than the likely range, so the listing may be a good price or may hide a problem with the car.

Comparison of models

Estimate for this car from each trained model, with each model's accuracy on 2,803 cars that were not used in training.

Model	Estimate	R ²	Avg. error (MAE)
XGBoost (final)	\$16,565	0.763	\$4,688
LightGBM	\$15,744	0.760	\$4,819
Random Forest	\$16,926	0.748	\$4,803
Ridge Regression	\$14,055	0.443	\$8,167

Figure 10. The application showing an estimate for a 2012 Hyundai sedan with an asking price.

Used Car Price Prediction

Enter the details of a used car to get an estimated market price in US dollars. Estimates are based on car listings from Georgia (the country) with production years 1980–2020.

Vehicle details

Fields marked * are required.

Vehicle

Manufacturer *
TOYOTA

Production year *
2024
Between 1980 and 2020
Production year must be between 1980 and 2020.

Body type *
Select...
Please select an option.

Colour *
Select...
Please select an option.

Engine and drivetrain

Engine volume (litres) *

0.5 to 7.0, for example 2.0
Engine volume is required.

Cylinders *

Cylinders is required.

☐ Turbocharged

Fuel type *
Select...
Please select an option.

Gear box *
Select...
Please select an option.

Drive wheels *
Select...
Please select an option.

Steering wheel *
Select...
Please select an option.

Condition and equipment

Estimate

Fill in the vehicle details and select "Estimate price" to see the result here.

Figure 11. Messages shown next to fields that are missing or out of range.

12.3 Running the System

From the project folder run `pip install -r requirements.txt` and then `python app.py`. The application opens at `http://127.0.0.1:5000` on the same computer. Instructions to repeat the training are in `README.md`.

13. System Testing and Results

13.1 Automated Tests

`tests/test_api.py` contains 11 tests that call the service the way the browser does. All 11 pass in about two seconds (command: `python -m unittest discover -s tests -v`).

Table 21. Automated tests

Test	What it checks
Valid prediction	A complete valid car returns a price within limits, a range around it, four model results and a notice that the levy default was used.
Every model selectable	Each of the four models can be chosen and is the one reported.
Unknown model rejected	An invalid model name returns status 400.
Missing required fields	Missing year, mileage and manufacturer each return their own message.
Invalid values rejected	24 invalid values across 9 fields (text in number fields, negative numbers, nan, infinity, values out of range, non-whole numbers, unknown fuel type or colour) are each rejected with status 400 and a message for that field.
Year range	1980 and 2020 are accepted; 2021 is rejected.
Turbo flag	The text value false behaves like false and does not change any other field.
Asking price check	A price far above and far below the estimate is reported as above and below the range; no asking price gives no check.
Bad request bodies	Broken JSON, a JSON list and a wrong HTTP method get a clear error and not a server crash.
Options and models	The option lists and the model list are complete and exactly one model is marked final.
Page renders	The home page loads.

13.2 Consistency Between Training and Serving

The most important check in Stage 9 was making sure the application gives the same answer as the training pipeline. We rebuilt the 2,803 test cars from the raw file using the same cleaning rules and the same split. Then we sent each car through the application's validation, feature building and model (`scripts/check_serving_consistency.py`), and compared the prices with the predictions made from the processed test file. All **2,803 predictions were identical** (largest difference \$0), and the validation rejected none of the valid test cars. This confirms the backend applies the same preprocessing as the training code.

13.3 Manual Tests of the Page

Table 22. Manual tests in the browser

Action	Expected	Observed
Press Estimate with an empty form	A message under every required field, no request sent, focus on the first wrong field	Passed: 14 messages shown and the focus moved to the manufacturer field.

Action	Expected	Observed
Enter production year 2024	Message that the year must be between 1980 and 2020	Passed.
Enter mileage -20	Message about the allowed range	Passed.
Enter a valid car without a levy	Estimate, range, model table and a notice about the levy	Passed.
Enter a valid car with an asking price	A sentence comparing the asking price and the estimate	Passed (Figure 10).
Choose another model	That model's estimate is the main figure	Passed: LightGBM selected, its estimate shown and named under the price.
View at a phone width (375 pixels)	Form and result stacked, no horizontal scrolling	Passed: one column, no horizontal scrolling.

13.4 Model Results in the System

The accuracy shown in the application's comparison table comes from `data/model_metrics.json`, which is computed from the saved models and the test split, so the page and this report quote the same numbers: for the final model R^2 0.7628, RMSE \$8,150 and MAE \$4,688.

14. Limitations and Possible Future Improvements

14.1 Limitations

- **Moderate accuracy.** The median error is 17%, and only 45% of estimates are within 15% of the real price. The likely range is wide (0.565 to 1.382 times the estimate), so the system should be used as a guide and not as a valuation.
- **Noisy prices.** The prices are asking prices set by sellers. Cheap listings, especially for recent cars, often don't follow the specification, and the model can't predict them (Table 12).
- **Few recent cars.** Only 1.6% of the cleaned records are from 2019 or 2020 and none is newer, so the application accepts only 1980 to 2020 and estimates for the newest cars are the least reliable.
- **Old market data.** Prices are those of the time the listings were collected. They are not corrected for inflation or for changes in the market since then, and the age is measured from 2026 while the prices belong to an earlier period.
- **One country, one marketplace.** The model should not be used for other markets.
- **No model or trim.** The car model name (for example Camry or Corolla) is not used because of its many rare values, although it clearly affects price.
- **Close competitors.** XGBoost, LightGBM and Random Forest can't be separated with confidence because of the fold-to-fold variation. A different random split could change their order.

- **Residual duplicates.** 73 test cars have the same feature values as a training car; the effect is small but not zero.
- **Local deployment.** The application runs on the user's own computer with Flask's built-in server; it is not hosted online.

14.2 Possible Improvements

- Collect more recent listings and a second source to reduce the noise and to cover new cars.
- Use the car model through a smarter encoding (for example grouping by make and body type, or target encoding inside cross-validation).
- Predict the range directly with quantile regression so that the interval depends on the car and not on one factor for all cars.
- Detect suspicious listings (for example a very cheap price for a new car) and remove them in a documented way.
- Explain each estimate to the user with feature contributions (for example SHAP values).
- Re-introduce a value-retention score or a deal rating once the error is small enough to support it.
- Host the service with a production web server, add logging and monitor the error against newly collected listings.

15. Individual and Group Contributions

The table lists the work of each member as recorded in the project notebook and the group's allocation file. All members took part in the discussion of the results and in the preparation of the report and presentation.

Table 23. Contributions by member

Member	Registration number	Contribution
N S Devapriya (Nirodha), team leader	IT23557956	Data cleaning engine and anomaly handling (Stage 4); XGBoost model; Flask service and integration of the models into the backend (Stage 9); coordination of the deliverables.
W G B S Wijerathna	IT23630734	Exploratory analysis of steering wheel side and of price against age and fuel type, and the correlation analysis (Stage 3); deal-class target (Stage 4); Random Forest model.
R A N K Ranasinghe	IT23780088	Target and mileage distribution analysis (Stage 3); feature engineering (Stage 4); LightGBM model.
T M A R Gunasekara	IT23614208	Train and test split and levy imputation, preprocessing pipeline (Stage 4); Ridge Regression and its tuning.

Stages 6 and 7 (model comparison, tuning and selection), Stage 10 (frontend), the testing and this report were completed as a group.

References

- Bergstra, J. and Bengio, Y. (2012). Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13, 281-305.
- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5-32.
- Chen, T. and Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785-794.
- Contractor, D. (n.d.). Car Price Prediction Challenge [Data set]. Kaggle.
<https://www.kaggle.com/datasets/deepcontractor/car-price-prediction-challenge>
- Hoerl, A. E. and Kennard, R. W. (1970). Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12(1), 55-67.
- Ke, G., Meng, Q., Finley, T., et al. (2017). LightGBM: A highly efficient gradient boosting decision tree. *Advances in Neural Information Processing Systems*, 30.
- Pallets Projects. (n.d.). Flask documentation. <https://flask.palletsprojects.com/>
- Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.